

SPIKE DASH

Especificação Técnica da Arquitetura Híbrida

Documento de Modelação de Dados e Fluxos Operacionais

Autor: BRUNO ANTONIO, MATHEUS SANTIAGO, WALLACE BARRETO

Docente: ÉRICLES DOS SANTOS CUNHA

1 Arquitetura Híbrida de Persistência

O ecossistema de mini-jogos competitivos do **Spike Dash** emprega uma infraestrutura híbrida de dados para equilibrar desempenho offline com sincronização online global. O fluxo de dados é segregado em três camadas:

Tabela 1: Divisão das Camadas de Armazenamento

Motor de Dados	Paradigma	Dados Geridos	Objetivo do Projeto
Hive	NoSQL Local	Configurações e Cache de Perfil	Acesso imediato em memória (latência
SQLite	Relacional Local	Histórico de partidas	Métricas analíticas (AVG, GROUP BY)
Cloud Firestore	NoSQL Nuvem	Rankings Globais e Usuários	Sincronização em tempo real via <i>Stream</i>

2 Dicionário de Dados e Schemas

2.1 Camada NoSQL Local (Hive)

Persistida de forma binária em caixas estruturadas de chave-valor.

- **Box configuracoes:**

- "darkMode" → bool (Default: true)
- "volumeMusic" → double (Default: 0.7)
- "volumeSfx" → double (Default: 1.0)

- **Box perfil_cache:**

- "uid", "nickname", "titulo" → String
- "nivel", "trofeus", "vitorias" → int
- "precisaoMedia" → double

2.2 Camada Relacional Local (SQLite)

Garante o histórico de partidas local em dispositivos nativos (Android/iOS).

```
CREATE TABLE historico_partidas(  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  modo_jogo TEXT NOT NULL,  
  pontuacao INTEGER NOT NULL,  
  precisao REAL NOT NULL,  
  combo_maximo INTEGER DEFAULT 0,  
  data_partida TEXT NOT NULL  
);
```

Listing 1: DDL da Tabela de Histórico de Partidas

2.3 Camada NoSQL na Nuvem (Cloud Firestore)

Representa o nó global e o barramento de sincronização do jogo.

- **Coleção usuarios/{uid}**: Atributos de cadastro e nível do jogador.
- **Coleção rankings/{uid}**: Documentos denormalizados ordenados para o Leaderboard global.
- **Coleção partidas/{uid}/historico/**: Espelho de segurança e histórico (essencial para persistência na Web).

3 Fluxos de Dados Principais

A gestão e alteração dos dados na interface gráfica são geridos de forma reativa através da biblioteca **Provider**.

3.1 Fluxo de Inicialização e Carregamento

CiclodeLeituraeSincronizaçãoInicial

1. **Fase1(Hive):** Aocarregaratela inicial,aUIconsomeocachedeperfil_cache imediatamente, exibindo o nickname e nível do jogador com zero latência.
2. **Fase2(Auth):** O AuthProvidere estabelece login assíncrono seguro com a nuvem utilizando o FirebaseAuth.
3. **Fase3(Firestore):** Mapeado o ID do utilizador, a aplicação lê o documento principal na coleção de usuarios/e atualiza os troféus vigentes obtidos de rankings/.
4. **Fase4(Atualização):** OcachelocaldoHiveéreescritocomosdadosmaisrecentes obtidos da nuvem.

3.2 Fluxo de Salvamento de Jogos (Fim de Partida)

O Spike Dash implementa uma abordagem dual de segurança para persistir pontuações, mitigando perdas mesmo quando ocorrem eventos de encerramento brusco de tela (dispose):

Mecanismo de Salvamento de Partida

Ao disparar o fim de jogo:

- **Caminho Padrão (Provider):** O jogo tenta salvar através do `context.read<UsuarioProvider>()` atualizando o estado reativo da aplicação.
- **Caminho de Fallback (Serviço Direto):** Se o contexto do widget for destruído durante o dispose, as instâncias Singletons globais (`HiveService`, `DatabaseLocal`, `FirestoreService`) assumem o fluxo, gravando localmente e agendando a sincronização assíncrona na nuvem.

4 Consultas Analíticas de Desempenho (SQLite)

O banco de dados relacional é usado localmente para gerar as estatísticas do jogador de forma consolidada e ágil:

```
-- Média de pontuação obtida em cada um dos modos de jogo
SELECT modo_jogo, AVG(pontuacao) as media_pontos
FROM historico_partidas
GROUP BY modo_jogo;

-- Melhor pontuação absoluta por modalidade de jogo
SELECT modo_jogo, MAX(pontuacao) as recorde
FROM historico_partidas
GROUP BY modo_jogo;
```

Listing 2: Consultas para Geração de Estatísticas

5 Regras de Segurança do Firestore

Para proteger as coleções contra modificações maliciosas, o servidor de nuvem valida as escritas com base na identidade autenticada do usuário:

```
rules_version='2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /usuarios/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth.uid == userId;
    }
    match /rankings/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth.uid == userId;
    }
  }
}
```

Listing 3: Regras de Validação do Banco Cloud

